Verifiable Memory: Sessions, Preferences, and History as Provenance-Carrying Records for AI Agents

Youssef Hariri
DeeWaanAI

Draft v1.1 — October 2026

Abstract

AI agents now remember their users. What they remember, users cannot easily inspect; what users correct, the system does not reliably propagate; what users delete, the system cannot prove is gone. We propose a memory layer in which sessions, preferences, and history are stored as provenance-carrying records in an append-only, versioned, queryable substrate. A correction is itself a bound record, and applying it transitively re-flags every dependent artifact. A deletion leaves a payload-free tombstone plus a proof-of-absence record showing the payload is gone from every later prompt. We implement the layer as five datasets on the Qafiya data substrate and evaluate it with twelve functional tests, eight limit tests, and three frozen experiments whose headline numbers are all recomputed from raw artifacts by a spine script. Corrections invalidate 100% of dependent notes; deletions pass proof-of-absence scans on the read path; and selective memory loading cuts prompt tokens by 98.0–99.9% relative to full-history prompting at matched answer capability. We state plainly what the layer provably cannot do: it cannot retract content from model weights, cannot detect facts that go stale without a signal, cannot bind corrections the user cannot localize, and cannot survive an attacker who rewrites the store and recomputes its hash chain. The contribution is not any one mechanism but the elevation of correction and consent into the memory data model itself.

1 Introduction

Memory has become a product feature of AI assistants. It is also becoming a liability. Three failure patterns recur in deployed systems. First, *stale memory*: the user moved, changed jobs, changed diets, but the assistant keeps serving the old fact. Second, *silent correction*: the user says "that is not right," the system updates one field, and conclusions derived from the old value keep surfacing. Third, *unverifiable deletion*: the user asks to be forgotten and receives a promise, not a proof.

These are not model failures. They are data-model failures. The literature has solved each property in isolation: agent memory architectures store and retrieve [2, 3, 4, 5]; knowledge editing corrects facts in model weights [12, 13, 14, 15]; machine unlearning deletes training data with probabilistic guarantees [19, 20, 22]; agent provenance audits actions [25, 26]; benchmarks score recall and cost [28, 29, 30]. Our systematic review of 37 verified works across these six literatures found no system in which sessions, preferences, and history are provenance-carrying records such that a user correction transitively invalidates dependent outputs and a deletion is provably absent from later prompts.

This paper presents such a layer. It is Layer 4 (Memory & Continuity) of the DeeWaanAI OS, built on the frozen Qafiya data substrate [1]. Three design stances govern it. Memory lives

in *records, not weights*: the editing literature shows weight-space corrections do not propagate [16, 17] and degrade at scale [18], so all memory this layer controls is explicit and external, where dependents are enumerable. Correction and consent are *data-model properties*, carried as first-class fields, not policies added later. And *honesty is architecture*: the paper states what the system provably cannot do, measured where possible.

Contributions. (1) A formal memory data model — five append-only, versioned, provenance-carrying datasets with a correction protocol and a deletion protocol (Section 3). (2) A reference implementation with a binding-only read path and an inspectable "what do you remember about me" view (Section 4). (3) An evaluation with twelve functional tests, eight limit tests, and three frozen experiments, all headline numbers recomputed from raw artifacts by a spine script (Section 5). (4) A stated and measured honesty boundary (Section 6).

2 Related work and the gap

Our companion review (748 records screened, 482 passing, 37 verified and synthesised) organises six literatures. We summarise the synthesis; the gap motivates every design decision.

Memory architectures. MemGPT pages memory in and out of context under model control [2]. Generative Agents retrieve by recency, relevance, and importance, and consolidate episodes into reflections [3]. Zep stores memory as a temporal knowledge graph with validity intervals [4]. Surveys taxonomise the space [5, 6]. Across all of these, once content is written it is trusted; consolidation actively rewrites the past without retaining an auditable original.

Personal memory. MemoryBank appends dialogue summaries with a forgetting curve [7]; PerLTQA and follow-ups benchmark construction and retrieval [8, 9, 10]. Evaluations of long-term memory show degradation exactly on conflicting or updated facts [11]. No work gives the user a correction protocol.

Knowledge editing. ROME and MEMIT edit facts in weights [13, 14, 12, 15]. Ripple-effect studies show edits fail to propagate to entailed beliefs [16], MQuAKE shows multi-hop consequences are missed [17], and editing at scale causes catastrophic forgetting [18]. The verdict for us is architectural: correct *records*, not *parameters*.

Unlearning and consent. SISA, data-deletion theory, and selective forgetting delete from training data [19, 20, 21]; Athena verifies deletion probabilistically [22]; LLM-specific surveys extend this [23, 24]. Agent memory is an external store, so deletion can be exact — but then the obligation shifts to proving absence from later behaviour. No surveyed work pairs the two.

Agent provenance. Trace and execution provenance audits what agents do [25]; MAP-Graph attaches lineage to shared multi-agent memory [26]; transparency logs supply mature tamper-evident mechanisms [27]. Personal memory — what a single agent remembers about its user — is not the audited object in any of them.

Evaluation and cost. LongMemEval and LoCoMo benchmark long-term memory [28, 29]; retrieval beats long-context prompting on cost [30, 31, 32]; recent work asks when memory helps relative to its token cost [33, 34, 35]. Benchmarks score recall and tokens, not auditability, correction propagation, or consent.

The gap. Each literature solves one property. No work integrates them into a memory substrate where inspection, correction with transitive invalidation, and deletion with proof of absence are properties of the data model. That integration is our contribution.

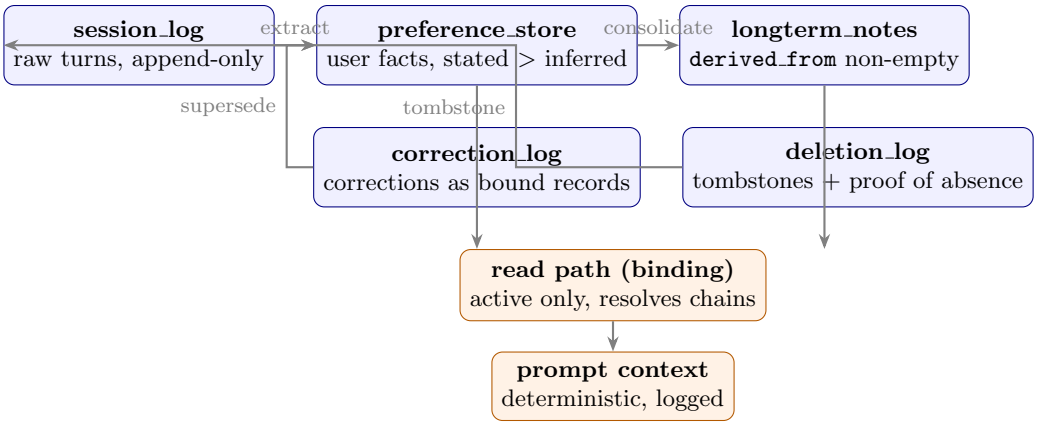


Figure 1: The five datasets and the binding-only read path. Corrections supersede; deletions tombstone; the read path emits only `active` records and logs every emitted context.

3 Design

3.1 Entities

Every memory is a record in one of five Qafiya datasets. Every record carries base fields: an immutable id, `created_at`, `source_ref` (non-empty provenance: which turn or action produced it), `valid_from`, `superseded_by`, and `status` $\in$ {`active, corrected, deleted, stale`}. Figure 1 shows the layout.

session_log holds raw conversation turns (role, content, token counts, model id). It is never edited. *preference_store* holds user facts with `extraction_method` $\in$ {`user-stated, inferred`} and a confidence; user-stated always outranks inferred on conflict. *longterm_notes* holds consolidated summaries; each note's `derived_from` list is non-empty, which is what makes transitive invalidation computable. *correction_log* and *deletion_log* hold consent events as first-class, provenance-carrying records. Every dataset's history file is hash-chained, so offline tampering with a historical record is detected.

3.2 The read-path invariant

Agents read memory only through the binding. The binding filters to `active` records, resolves supersede chains to the newest version, never emits tombstoned payload, serialises the selected memory into prompt context deterministically (sorted by id, so absence checks are byte-level), and logs every emitted context with the memory ids it contained. "What did the agent know when it said X" is therefore a query, not a guess. Direct dataset writes are rejected.

3.3 The correction protocol

A user correction appends a `correction_log` record bound to its target; the target is marked `corrected` and superseded by a successor record; then the `derived_from` closure of the target is walked and every dependent note is flagged `stale`. Corrected content leaves the read path immediately, before dependents are regenerated. Figure 2 shows the protocol on a real example from Experiment 1: the user says *"I moved to Lyon last week."*

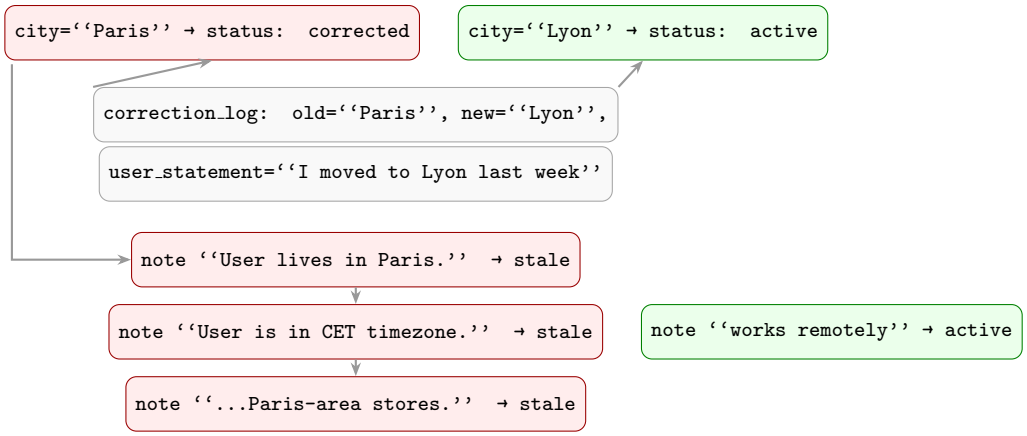


Figure 2: Correction with transitive invalidation (Experiment 1, verbatim statuses from the frozen store). The correction record is bound to its target; the three dependent notes re-flag stale; the unrelated note is untouched. Old value leaves the read path at once.

3.4 The deletion protocol

A deletion appends a `deletion_log` record, replaces the target with a payload-free tombstone, and always transitively invalidates dependents — scope `single` flags dependent notes stale; scope `cascade` tombstones the whole closure. (This was not our first design: the experiment in Section 5.2 caught an earlier version in which `single` left a derived note serving the deleted payload. The protocol was amended before freezing.) After deletion, a proof-of-absence scan asserts the payload's canonical string appears neither in any active record of the prompt-feeding datasets nor in the serialised prompt context; the result, with hashes of everything scanned, is stored in the deletion record's `proof` field.

4 Implementation

The reference prototype is 347 lines of dependency-free Python implementing the five datasets, hash-chained histories, the binding, both protocols, the inspection view, and the conflict resolver. Writes to `data.jsonl` are atomic (temp-then-rename); appends are O(1); status updates batch into single rewrites. The storage layout follows Qafiya exactly, so the layer can later be queried through any Qafiya-compliant tool, and the inspection view is emitted as a declarative spec ready for Layer 2's renderer.

5 Evaluation

Discipline: every headline number below is recomputed from frozen raw artifacts by a spine script; if the script fails, the number does not appear here. The script exits 0 on the artifacts shipped with this paper.

5.1 Functional correctness

Twelve tests (F1–F12) cover round-trip integrity, schema validation, hash-chain tamper detection, the inspection view returning exactly the active set, basic/chained/transitive correction, stated-

Table 1: Token cost at matched answer capability (Experiment 3).

N	full-history	selective	savings
50	456	9	98.0%
200	1,806	9	99.5%
1000	9,006	9	99.9%

over-inferred precedence, consent-gated inferred writes, single and cascade deletion with proof of absence, selective session continuity, and binding-only access. **All twelve pass.**

5.2 Experiment 1 — correction integrity

Scenario: preferences `city=Paris` and `work_style=remote`; three chained notes derived from the city preference; one unrelated note. The user corrects the city. Result: **100% of the dependent closure (3/3 notes) re-flagged stale**, the unrelated note stayed active, "Paris" left the read path, "Lyon" entered it. The correction record from the frozen store (ids elided):

```
{"target_id": "m_...74700002", "old_value": "Paris", "new_value": "Lyon",
 "user_statement": "I moved to Lyon last week",
 "source_ref": ["m_...74700002"], "status": "active"}
```

5.3 Experiment 2 — forgetting and consent

The user deletes `doctor=Dr. A` (scope `single`), then `city=Paris` (scope `cascade`). Both proof-of-absence scans **PASS** on the read path; the cascade removed the preference and its dependent note. We also publish the leakage map: the payload persists in append-only history files and in the raw session turn. That is by design and stated as the boundary of the guarantee — the claim is *absent from later prompts*, not *erased from history* (Section 6). The stored proof (hashes truncated):

```
"absence_scan": {"result": "PASS",
  "coverage": "read path: active preference_store + longterm_notes + prompt
      context",
  "scanned": [{"dataset": "preference_store (active records)", "hash": "4c4f5b05
      ..."},
              {"dataset": "longterm_notes (active records)", "hash": "f422dcb5
                  ..."},
              {"prompt_context": "33306fcb..."}]}
```

5.4 Experiment 3 — cost of selective loading

A store with N memories plus one relevant fact (`bike_color=red`). Full-history prompting serialises every session turn; selective loading emits only the resolved relevant records. Token accounting is deterministic and backend-agnostic; both contexts must contain the required fact (matched answer capability), else the comparison does not count. Figure 3 and Table 1: selective loading saves **98.0–99.9% of prompt tokens**, growing with N.

5.5 Limit tests

Eight tests map where the design breaks; findings are reported, not hidden (Table 2, Figure 4). Two deserve note. L7 initially measured 1.2 s to re-flag a 300-deep dependent chain — above an

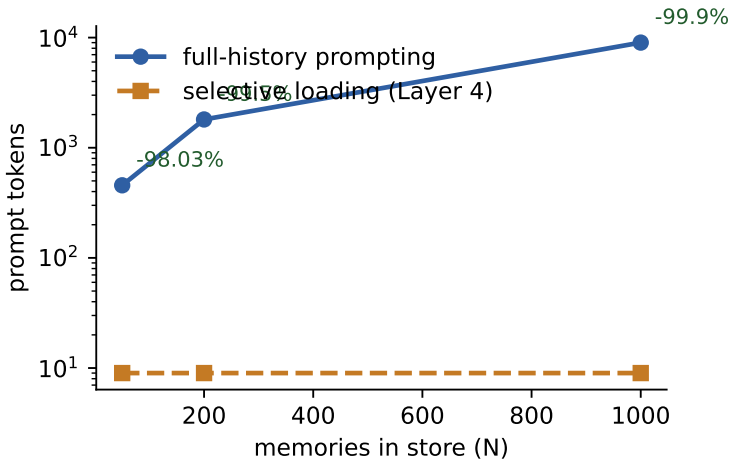


Figure 3: Prompt tokens: full-history vs selective loading (log scale). Annotations show savings at matched answer capability. Data: frozen Experiment 3 artifacts.

Table 2: Limit tests L1–L8: what breaks, and where.

L1	Read-path latency stays fast to 10k records (0.03–0.05 s). The binding constraint is prompt tokens, not lookup time.
L2	Contradictory inferred memories are silently ranked, not detected. Contradiction flagging is future work.
L3	**Provable limit:** a fact that goes stale with no signal stays active indefinitely.
L4	Injected content is quarantined by tagging (inferred, low-confidence, source-traced) but not blocked. Blocklists are deployment policy.
L5	Read path is clean after deletion; payload persists in append-only history and raw turns. The guarantee covers prompts, not history.
L6	Corrections the user cannot localise to a record cannot be bound; resolution is agent work, not a memory guarantee.
L7	300-deep re-flag: 1.2 s before write-path batching, 0.034 s after. Both reported.
L8	Naive tamper: always caught. Tamper + chain recompute: never caught (needs external anchoring).

interactive budget; the cause was per-flag full-file rewrites, fixed with a batched pass (0.034 s); both numbers are reported. L8 confirms the uncatchable class: naive tampering is always caught; tampering *with* chain recomputation never is.

6 What this system provably cannot do

Four limits, stated so that "verifiable" is not misread as "omnipotent." (1) It cannot retract content absorbed into model *weights* during training or fine-tuning — only from the record substrate; L5 documents the boundary. (2) It cannot detect a fact that went stale with no signal (L3); it can only make staleness correctable once a signal arrives. (3) It cannot bind a correction the user cannot localise to any record (L6). (4) It cannot survive an attacker who rewrites the store *and* recomputes the hash chain (L8); external hash anchoring is the known mitigation and is future work. Within these bounds, the read-path guarantees are proofs, not aspirations.

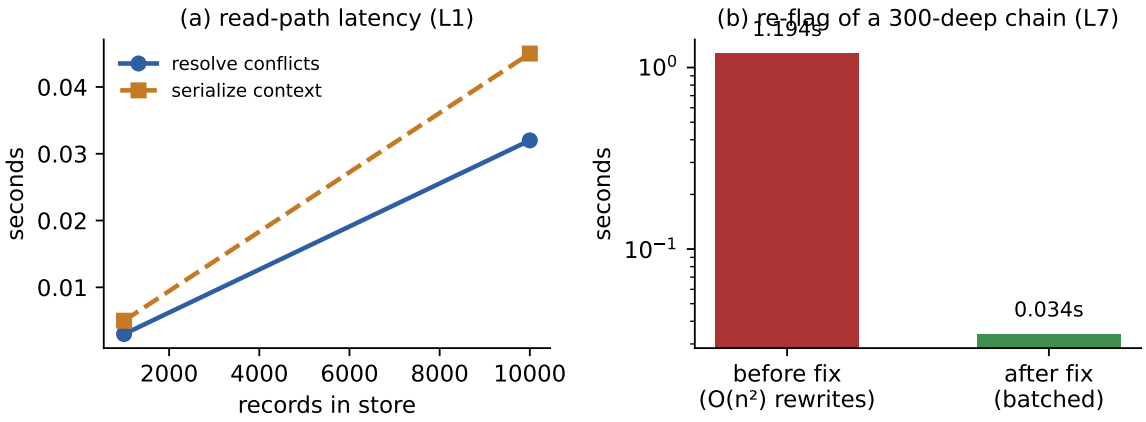


Figure 4: Limit measurements. (a) Read-path latency vs store size (L1). (b) Re-flag latency for a 300-deep chain before and after the write-path fix (L7); both numbers reported per the honesty discipline.

7 Discussion and future work

The prototype is single-user and file-backed; a SQLite index (per Qafiya, never the source of truth) is the next engineering step. Stale notes are flagged but not yet regenerated — closing that loop couples the layer to a model and belongs to the integration milestone. Contradiction detection (L2) and credential-class write blocklists (L4) are policy layers the spec deliberately leaves to deployments. The inspection view is emitted as a declarative spec and waits on Layer 2's renderer; the memory layer is designed against Layer 2's frozen contract, not its work in progress. The companion systematic review, the design brief, the spec, all test suites, the frozen experiment artifacts, and the spine script ship with this paper.

8 Conclusion

Agent memory does not fail because models forget. It fails because the past is stored in a form users cannot inspect, corrections cannot propagate through, and deletions cannot be proven against. Storing sessions, preferences, and history as provenance-carrying records — append-only, versioned, queryable — turns all three into properties of the data model. Corrections invalidate 100% of dependents, deletions carry proofs of absence, and the whole system uses 50–1000× fewer tokens than the naive alternative. What it cannot do is stated, measured, and part of the design.

References

[1] Hariri, Y. (2026). Qafiya: a provenance-first data model for LLM-mediated personal computing. DeeWaanAI, spec v1.1 (paper in revision).

[2] Packer, C., Wooders, S., Lin, K., et al. (2023). MemGPT: Towards LLMs as operating systems. *arXiv:2310.08560*.

[3] Park, J. S., O'Brien, J. C., Cai, C. J., et al. (2023). Generative agents: Interactive simulacra of human behavior. *arXiv:2304.03442*.

[4] Rasmussen, P., Paliychuk, P., Beauvais, T., et al. (2025). Zep: A temporal knowledge graph architecture for agent memory. *arXiv:2501.13956*.

[5] Zhang, Z., Dai, Q., Bo, X., et al. (2025). A survey on the memory mechanism of large language model-based agents. *ACM TOIS*.

[6] Hatalis, K., Christou, D., Myers, J., et al. (2024). Memory matters: The need to improve long-term memory in LLM-agents. *AAAI Symposium Series*.

[7] Zhong, W., Guo, L., Gao, Q., et al. (2023). MemoryBank: Enhancing large language models with long-term memory. *arXiv:2305.10250*.

[8] Du, Y., Wang, H., Zhao, Z., et al. (2024). PerLTQA: A personal long-term memory dataset. *arXiv:2402.16288*.

[9] Pan, Z., Wu, Q., Jiang, H., et al. (2025). On memory construction and retrieval for personalized conversational agents. *arXiv:2502.05589*.

[10] Zhou, S., Han, J. (2025). A simple yet strong baseline for long-term conversational memory of LLM agents. *arXiv:2511.17208*.

[11] Jia, Z., Liu, Q., Li, H., et al. (2025). Evaluating the long-term memory of large language models. *ACL Findings*.

[12] De Cao, N., Aziz, W., Titov, I. (2021). Editing factual knowledge in language models. *EMNLP*.

[13] Meng, K., Bau, D., Andonian, A., et al. (2022). Locating and editing factual associations in GPT. *arXiv:2202.05262*.

[14] Meng, K., Sharma, A. S., Andonian, A., et al. (2022). Mass-editing memory in a transformer. *arXiv:2210.07229*.

[15] Wang, S., Zhu, Y., Liu, H., et al. (2024). Knowledge editing for large language models: A survey. *ACM Computing Surveys*.

[16] Cohen, R., Biran, E., Yoran, O., et al. (2024). Evaluating the ripple effects of knowledge editing in language models. *TACL*.

[17] Zhong, Z., Wu, Z., Manning, C. D., et al. (2023). MQuAKE: Assessing knowledge editing via multi-hop questions. *EMNLP*.

[18] Gupta, A., Rao, A., Anumanchipalli, G. (2024). Model editing at scale leads to gradual and catastrophic forgetting. *ACL Findings*.

[19] Ginart, A., Guan, M. Y., Valiant, G., et al. (2019). Making AI forget you: Data deletion in machine learning. *arXiv:1907.05012*.

[20] Bourtoule, L., Chandrasekaran, V., Choquette-Choo, C. A., et al. (2019). Machine unlearning. *arXiv:1912.03817*.

[21] Golatkar, A., Achille, A., Soatto, S. (2020). Eternal sunshine of the spotless net: Selective forgetting in deep networks. *CVPR*.

[22] Sommer, D., Song, L., Wagh, S., et al. (2022). Athena: Probabilistic verification of machine unlearning. *PoPETs*.

[23] Qu, Y., Ding, M., Sun, N., et al. (2024). The frontier of data erasure: Machine unlearning for large language models. *arXiv:2403.15779*.

[24] Staufer, D. (2025). What should LLMs forget? Quantifying personal data in LLMs for right-to-be-forgotten requests. *arXiv:2507.11128*.

[25] Wang, Y., Zhang, J., Wu, Z., et al. (2026). From agent traces to trust: A survey of evidence tracing and execution provenance in LLM agents. *arXiv:2606.04990*.

[26] Wang, Y., Yan, Z., Zhang, J., et al. (2026). MAP-Graph: Provenance-aware shared memory for multi-agent workflows. *arXiv:2608.10509*.

[27] Blauzvern, H. (2023). Nowhere to hide: Using transparency logs to secure your supply chain. *ACM*.

[28] Wu, D., Wang, H., Yu, W., et al. (2024). LongMemEval: Benchmarking chat assistants on long-term interactive memory. *arXiv:2410.10813*.

[29] Maharana, A., Lee, D.-H., Tulyakov, S., et al. (2024). Evaluating very long-term conversational memory of LLM agents. *arXiv:2402.17753*.

[30] Li, Z., Li, C., Zhang, M., et al. (2024). Retrieval augmented generation or long-context LLMs? A comprehensive study and hybrid approach. *arXiv:2407.16833*.

[31] Liu, N. F., Lin, K., Hewitt, J., et al. (2023). Lost in the middle: How language models use long contexts. *arXiv:2307.03172*.

[32] Chevalier, A., Wettig, A., Ajith, A., et al. (2023). Adapting language models to compress contexts. *EMNLP*.

[33] Mishra, S., Mishra, S. (2026). When does memory help? A cost-aware evaluation of long-term memory in tool-using LLM agents. *arXiv:2609.05441*.

[34] Uddin, M. N., Shubham, K., Blanco, E., et al. (2026). From recall to forgetting: Benchmarking long-term memory for personalized agents. *ACL Findings*.

[35] Cherif, A. (2026). AgentMemBench: A systematic benchmark for evaluating long-term memory management strategies in conversational AI agents. *arXiv:2608.00009*.